Descripció

Partint d'alguns arxius CSV dissenyaràs i crearàs la teva base de dades.

Important

Totes les transformacions i importacions necessàries en aquesta tasca s'han de fer **mitjançant codi SQL. NO ES PERMET utilitzar el Wizard. Cal descriure cada pas i fer una captura de pantalla.**

Nivell 1

Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:

Primer de tot, el que fem és crear la base de dades a la qual anomenem **sprint4_db** on es carregaran els arxius CSV, usant la sentència **SQL CREATE DATABASE**, afegint la sentència **CHARACTER SET utf8mb4** que significa configurar el conjunt de caràcters per admetre Unicode complet.

```
7 •  CREATE DATABASE IF NOT EXISTS sprint4_db CHARACTER SET utf8mb4;
8 •  USE sprint4_db;
9
10
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:12:08	CREATE DATABASE IF NOT EXISTS sprint4_db CHARACTER SET utf8mb4	1 row(s) affected	0.000 sec

Havent analitzat les diferents taules facilitades en l'arxiu CSV, les taules i les relacions entre elles que es carregaran a la base de dades per obtenir el model estrella requerit, són les següents:

- **transactions** (taula de fets): És el centre del model estrella, ja que conté els esdeveniments quantitatius de la base de dades. Té relació N:1 amb tota la resta de les taules de dimensions (companyies, credit_cards, i productes) a través de les variables **business_id, card_id i user_id** que definirem com a clau forana d'aquestes tres taules. Hi ha l'excepció inicialment, de la taula "**products**", ja que la variable "**product_ids**" conté diferents productes i no podem establir la relació.
- **products** (taula de dimensió): dades dels productes amb la variable "**product_id**" com a PK. No es pot establir relació amb la taula de fets "**transactions**"
- **companies** (taula de dimensió): dades de les empreses amb variable "**id_company**" com a PK. Relació 1:N amb taula de fets "**transactions**"
- **credit_cards** (taula de dimensió): dades de les targetes de crèdit amb variable "**id_credit card**" com a PK. Relació 1:N amb taula de fets "**transactions**"

- **users** (taula d dimensió): dades dels usuaris amb variable **id_user** com a PK. Carregarem les dues taules "european_user" i "american_users" en la taula users. Relació 1:N amb taula de fets "**transactions**"

Creació de taules

Taula "**products**":

```
CREATE TABLE products (
    product_id VARCHAR (20) PRIMARY KEY,
    product_name  VARCHAR(100),
    price VARCHAR(20),
    colour VARCHAR(20),
    weight VARCHAR(20),
    warehouse_id VARCHAR(20)
);
```

Taula "**users**":

```
CREATE TABLE IF NOT EXISTS users (
    id_user VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100),
    surname VARCHAR(100),
    phone VARCHAR(150),
    email VARCHAR(150),
    birth_date VARCHAR(50),
    country VARCHAR(150),
    city VARCHAR(150),
    postal_code VARCHAR(100),
    address VARCHAR(255)
);
```

Taula "**credit_cards**":

```
-- Creem taula credit_cards
CREATE TABLE IF NOT EXISTS credit_cards  (
    id_credit_card VARCHAR(20) PRIMARY KEY,
    user_id INT,
    iban VARCHAR(34),
    pan VARCHAR(20),
    pin VARCHAR(4),
    cvv VARCHAR(3),
    track1 VARCHAR(255),
    track2 VARCHAR(255),
    expiring_date VARCHAR(20)
);
```

Taula "**companies**":

```
CREATE TABLE IF NOT EXISTS companies (
    id_company VARCHAR(20) PRIMARY KEY,
    company_name VARCHAR(100),
    phone VARCHAR(25),
    email VARCHAR(100),
    country VARCHAR(50),
    website VARCHAR(100)
);
```

Taula "**transactions**":

```
## creem la taula de transaccions
CREATE TABLE IF NOT EXISTS transactions (
    id_transaction VARCHAR(255) PRIMARY KEY,
    card_id VARCHAR(20),
    business_id VARCHAR(255),
    timestamp DATETIME,
    amount DECIMAL(10,2),
    declined BOOLEAN,
    product_ids VARCHAR(100),  -- ids seguits, no podem fer la FK
    user_id VARCHAR(20),
    lat FLOAT,
    longitude FLOAT
    );
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	20:20:44	CREATE TABLE IF NOT EXISTS transactions (id_transaction VARCHAR(255) PRIMARY KEY, card_id VARCHAR(20), business_id VARCHAR(2...	0 row(s) affected	0.094 sec

Un cop creades les taules, abans de carregar les dades, el que fem és executar la instrucció **SET GLOBAL local_infile = 1**; que habilita en MYSQL la capacitat del servidor per importar fitxers en local, perquè sinó dona error "*Error Code: 3948. Loading local data is disabled; this must be enabled on both the client and server sides in*"

```
SET GLOBAL local_infile = 1;
SHOW VARIABLES LIKE 'local_infile';
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Variable_name	Value
local_infile	ON

Result 3 — Read Only

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	07:23:57	SET GLOBAL local_infile = 1	0 row(s) affected	0.000 sec
2	07:24:02	SHOW VARIABLES LIKE 'local_infile'	1 row(s) returned	0.016 sec / 0.000 sec

<u>Càrrega de dades dels fitxers CSV a les taules de la BBDD</u>

A partir d'aquí hem anat fent la importació de taules CSV, amb el següent codi SQL:

LOAD DATA LOCAL IN FILE: Serveix per importar dades des d'un fitxer posat en la ruta local indicada.

INTO TABLE: la taula on importem les dades.

FIELDS TERMINATED BY',' : com estan separades les dades de l'arxiu,
ENCLOSED BY '"': que els valors estan entre cometes.

LINES TERMINATED BY '\n':com finalitza cada fila de l'arxiu i

IGNORE 1 ROWS: per indicar que no faci cas de la primera línia que és la capçalera de la taula.

Addicionalment, si en alguna taula fem la modificació de tipus o alguna altra transformació d'alguna dada importada, posarem al final del codi descrit prèviament, totes les columnes de la capçalera de la taula i amb @ a la variable a modificar, que serà guardada en una variable temporal mentre es transforma.

Taula “**products**”: En la carrega transformem la variable “price” a tipus decimal, eliminant el símbol “$” que hi ha en les dades del CSV, ho fem amb la funció REPLACE i el canvi de tipus amb la funció “CAST”: **SET price = CAST(REPLACE(@price, '$', '') AS DECIMAL(10,2));**

```
LOAD DATA LOCAL INFILE 'C:/Users/jaume/products.csv'
INTO TABLE products
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(product_id, product_name, @price, colour, weight, warehouse_id)
SET price = CAST(REPLACE(@price, '$', '') AS DECIMAL(10,2)); -- traiem el símbol del "$" en variable "price" i li assignem tipus decimal. La dada introduïda en @price és una variable temporal

-- Visualitzem les dades carregades
SELECT *
FROM products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	product_name	price	colour	weight	warehouse_id
1	Direwolf Stannis	161.11	#7c7c7c	1	WH-4
10	Karstark Dorne	119.52	#f4f4f4	2.4	WH-5
100	south duel	40.43	#6d6d6d	3	WH-95
11	Karstark Dorne	49.70	#141414	2.7	WH-6
12	duel Direwolf	181.60	#a8a8a8	2.1	WH-7
13	palpatine chewbacca	139.59	#2b2b2b	1	WH-8
14	Direwolf	147.53	#c4c4c4	2	WH-9
15	Stannis warden	194.29	#dbdbdb	1.5	WH-10
16	the duel warden	180.91	#666666	3	WH-11
17	skywalker ewok sith	91.89	#7c7c7c	3.2	WH-12
18	Karstark warden	148.91	#c4c4c4	0.8	WH-13
19	dooku solo	60.33	#3f3f3f	0.6	WH-14
2	Tarly Stark	9.24	#919191	2	WH-3
20	warden Karstark	91.96	#b5b5b5	1.4	WH-15
21	duel Direwolf	96.90	#e2e2e2	1.2	WH-16
22	chewbacca mustafar	150.02	#fcfcfc	2.4	WH-17
23	riverlands north	169.96	#545454	2.7	WH-18
24	south duel tourney	48.99	#aaaaaa	2.1	WH-19

products 8

Output

Action Output

#	Time	Action	Message	Duration /
1	08:04:31	LOAD DATA LOCAL INFILE 'C:/Users/jaume/products.csv' INTO TABLE products FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES TERMI...	100 row(s) affected Records: 100 Deleted: 0 Skipped: 0 Warnings: 0	0.032 sec
2	08:04:35	SELECT * FROM products LIMIT 0, 50000	100 row(s) returned	0.000 sec

Taula “**Users**”: Hi carreguem les dades dels fitxers “european_users” i “american_users”. En la càrrega, transformem la variable “**birth_date**” a format DATE

```
LOAD DATA LOCAL INFILE 'C:/Users/jaume/american_users.csv'
INTO TABLE dim_user
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS -- omissió del primer registre que normalment son els encapçalament de les columnes.
(id_user, name, surname, phone, email, @birth_date, country, city, postal_code, address)
SET birth_date = STR_TO_DATE(@birth_date, '%b %e, %Y');
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	07:48:42	LOAD DATA LOCAL INFILE 'C:/Users/jaume/american_users.csv' INTO TABLE dim_user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES ...	1010 row(s) affected Records: 1010 Deleted: 0 Skipped: 0 Warnings: 0	0.063 sec

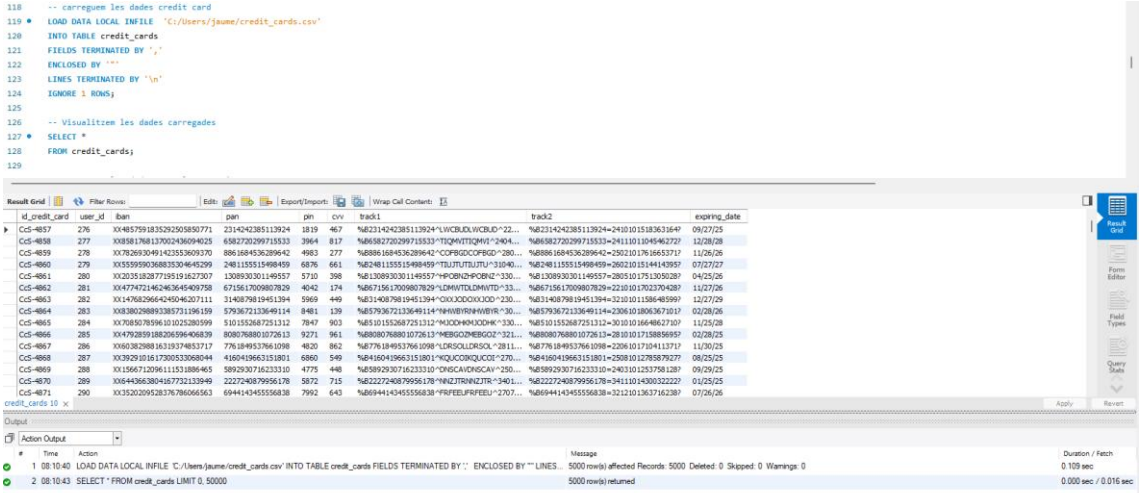
```
LOAD DATA LOCAL INFILE 'C:/Users/jaume/european_users.csv'
INTO TABLE dim_user
FIELDS TERMINATED BY ','
ENCLOSED BY '"'
LINES TERMINATED BY '\n'
IGNORE 1 ROWS
(id_user, name, surname, phone, email, @birth_date, country, city, postal_code, address)
SET birth_date = STR_TO_DATE(@birth_date, '%b %e, %Y');
```

Output

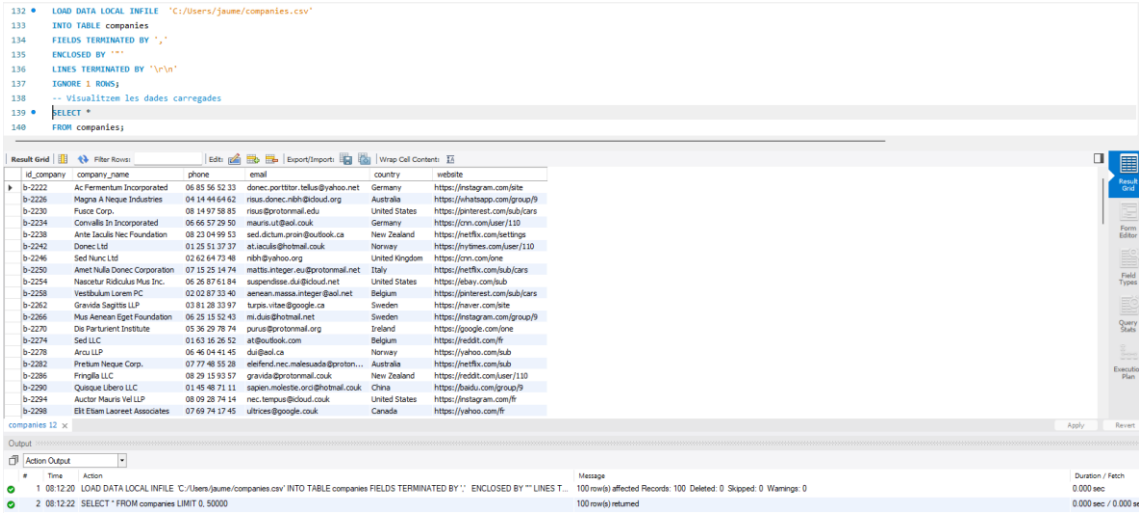
Action Output

#	Time	Action	Message	Duration / Fetch
1	07:50:17	LOAD DATA LOCAL INFILE 'C:/Users/jaume/european_users.csv' INTO TABLE dim_user FIELDS TERMINATED BY ',' ENCLOSED BY '"' LINES ...	3990 row(s) affected Records: 3990 Deleted: 0 Skipped: 0 Warnings: 0	0.078 sec

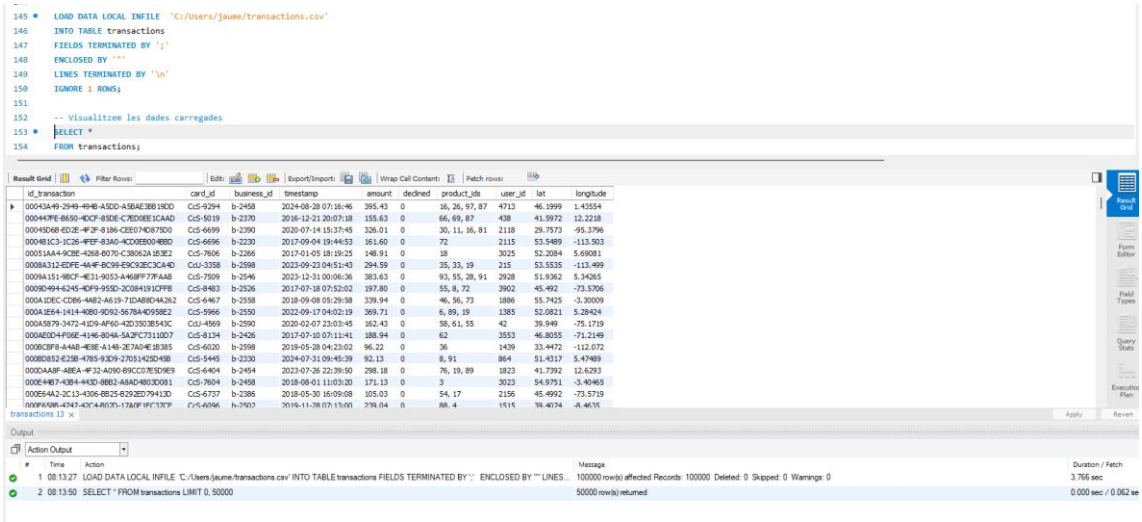
Taula “**credit_cards**”:



Taula “**companies**”:



Taula “**transactions**”:



Relacions entre taules

Establim les relacions descrites en l'apartat inicial. La taula de "**transaccions**" té relació N:1 amb tota la resta de les taules de dimensions, menys la de "products", a través de les variables **business_id, card_id i user_id** que definim com a clau forana de les taules **credit_cards, companies i users**.

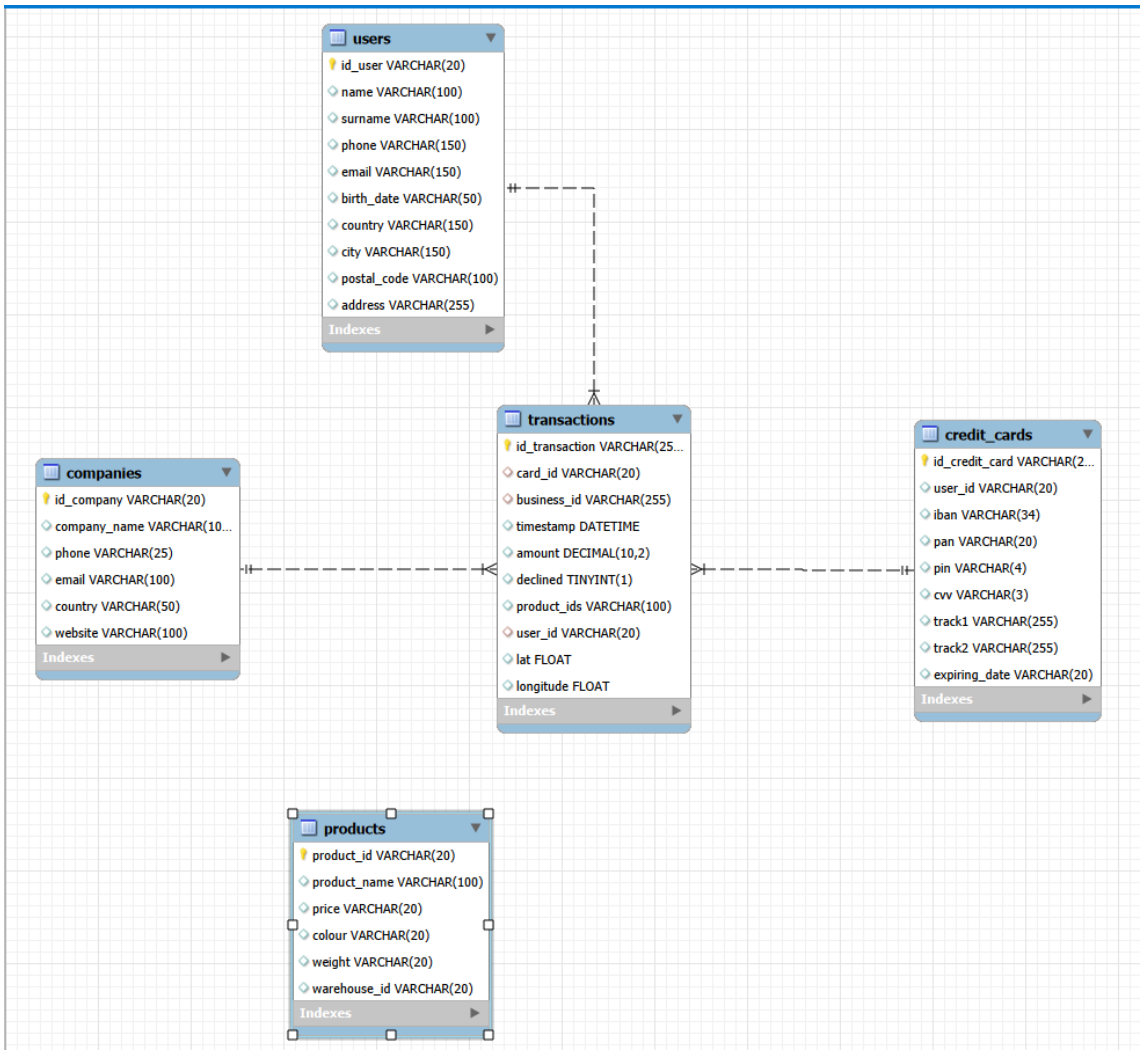
```
-- Afegim les relacions entre les taules a través de les Fk

ALTER TABLE transactions
ADD CONSTRAINT fk_transactions_credit_cards
FOREIGN KEY (card_id)
REFERENCES credit_cards(id_credit_card);

ALTER TABLE transactions
ADD CONSTRAINT fk_transactions_companies
FOREIGN KEY (business_id)
REFERENCES companies (id_company);

ALTER TABLE transactions
ADD CONSTRAINT fk_transactions_users
FOREIGN KEY (user_id)
REFERENCES users (id_user);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	08:23:16	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_credit_cards FOREIGN KEY (card_id) REFERENCES credit_cards(id_credit_card)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	2.891 sec
2	08:23:30	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_companies FOREIGN KEY (business_id) REFERENCES companies (id_company)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	3.328 sec
3	08:23:43	ALTER TABLE transactions ADD CONSTRAINT fk_transactions_users FOREIGN KEY (user_id) REFERENCES users (id_user)	100000 row(s) affected Records: 100000 Duplicates: 0 Warnings: 0	4.250 sec

Finalment posem captura del model resultant:



Exercici 1

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

Fem servir l'operador EXISTS per veure quins usuaris de la taula **users** compleixen el fet d'haver realitzat més de 80 transaccions. Això ho fem en la subconsulta, fent un GROUP BY de l'usuari i mirant si n'ha fet més de 80 en el HAVING

```
## Exercici 1
## Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.


SELECT u.id_user, u.name, u.surname
FROM users u
WHERE EXISTS (
    SELECT 1
    FROM transactions t
    WHERE t.user_id = u.id_user AND t.declined =0
    GROUP BY t.user_id
    HAVING COUNT(t.id_transaction) > 80
);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

id_user	name	surname
185	Molly	Gilliam
289	Dxwgi	Hwcru
318	Bmyr	Astuw
NULL	NULL	NULL

users 19 | Apply | Revert

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	08:26:16	SELECT u.id_user, u.name, u.surname FROM users u WHERE EXISTS (SELECT 1 FROM transactions t WHERE t.user_id = u.id_user A...	3 row(s) returned	0.578 sec / 0.000 sec

Exercici 2

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Fem dues INNER JOIN per poder unir la taula "**transactions**" amb taula "**companyies**" mitjançant l'ID de l'empresa per poder filtrar pel seu nom (Donec Ltd) i taula "**transactions**" amb "**credit_cards**" per poder associar cada transacció amb un IBAN concret. Fem un GROUP BY per agrupar per IBAN i un ORDER BY DESC per ordenar pels IBAN's amb mitjana més gran.

Nivell 2

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en *si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu*. Partint d'aquesta taula respon:

Exercici 1

Quantes targetes estan actives?

Creem una taula anomenada **estat_targetes** que tindrà dues variables "**card_id**" i "**estat_targetes**" que vindran donats per un consulta que retorna un taula amb aquests dos valors.

La consulta classifica cada targeta segons es demana en l'enunciat (Activa i Inactiva). En aquesta, es realitza una subconsulta interna per ordenar les transaccions de cada targeta, de la més recent a la més antiga, mitjançant la instrucció **ORDER BY ft.timestamp DESC dins la funció ROW_NUMBER()** que serveix per numerar les transaccions i posteriorment, en el WHERE es filtra per tres les últimes transaccions fetes.

Mirem quantes d'aquestes tres transaccions han estat rebutjades (declined = 1). Si les tres últimes han estat rebutjades la suma serà 3 i qualificarem la variable "estat_targeta" com a "Inactiva", si es menor a 3, "estat_targeta" serà activa.

```sql
CREATE TABLE estat_targetes AS -- El contingut de la taula vindrà donat per la següent consulta
SELECT
    card_id,
    CASE
        WHEN SUM(CASE WHEN declined = 1 THEN 1 ELSE 0 END) = 3 THEN 'Inactiva' -- si dona 3 rebutjades posem en inactiva, sino la targeta serà activa
        ELSE 'Activa'
    END AS estat_targeta
FROM (
    SELECT
        t.card_id,
        t.declined,
        ROW_NUMBER() OVER (PARTITION BY t.card_id ORDER BY t.timestamp DESC) AS filtra_transacció -- Ordena les transaccions de cada targeta per data més recent
    FROM transactions t
) ultimes_transaccions
WHERE filtra_transacció <= 3 -- ens quedem amb com a màxim 3 transaccions per targeta que seran les 3 més recents per tal com s'han ordenat
GROUP BY card_id;

-- Visualitzem la taula creada
SELECT *
FROM estat_targetes;
```

card_id	estat_targeta
Cc5-4857	Activa
Cc5-4858	Activa
Cc5-4859	Activa
Cc5-4860	Activa
Cc5-4861	Activa
Cc5-4862	Activa

#	Time	Action	Message	Duration / Fetch
1	08:32:00	CREATE TABLE estat_targetes AS -- El contingut de la taula vindrà donat per la següent consulta SELECT card_id, CASE WHEN SUM(C...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.687 sec
2	08:32:07	SELECT * FROM estat_targetes LIMIT 0, 50000	5000 row(s) returned	0.000 sec / 0.000 sec

A partir de la taula creada, fem la consulta per veure quantes targetes estan "Actives"

```sql
#Finalment, a partir de la taula creada, fem la consulta per veure quantes targetes estan "Actives"
SELECT estat_targeta, COUNT(*) AS total_targetes
FROM estat_targetes
GROUP BY estat_targeta
HAVING estat_targeta ="Activa";
```

estat_targeta	total_targetes
Activa	4995

#	Time	Action	Message
1	08:46:00	SELECT estat_targeta, COUNT(*) AS total_targetes FROM estat_targetes GROUP BY estat_targeta HAVING estat_targeta ="Activa" LIMIT 0, 50000	1 row(s) returned

A la taula creada "**estat_targetes**", definim a la variable "credit_card" com a PK i la relacionem amb la taula "**credit_cards**" fent que sigui FK de la variable PK de "**credit_cards**", "**id_credit_card**"

```sql
-- Per afegir al model la taula "estat_targetes" assignem a la variable "card_id" de la taula com PK i FK de la taula "Credit_cards
ALTER TABLE estat_targetes ADD PRIMARY KEY (card_id);

ALTER TABLE estat_targetes
ADD CONSTRAINT fk_estat_targetes_credit_cards
FOREIGN KEY (card_id)
REFERENCES credit_cards(id_credit_card);
```

#	Time	Action	Message	Duration / Fetch
1	12:17:41	ALTER TABLE estat_targetes ADD PRIMARY KEY (card_id)	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	0.218 sec
2	12:17:44	ALTER TABLE estat_targetes ADD CONSTRAINT fk_estat_targetes_credit_cards FOREIGN KEY (card_id) REFERENCES credit_cards(id_credit_c...	5000 row(s) affected Records: 5000 Duplicates: 0 Warnings: 0	0.078 sec

Nivell 3

Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

Exercici 1

Necessitem conèixer el nombre de vegades que s'ha venut cada producte.

Per resoldre el que se’ns planteja a l’exercici, i poder respondre a la pregunta de "quants cops s'ha venut cada producte", creem una taula intermèdia que anomenem “**transactions_products**” que relaciona la taula de fets “**transactions**” i la taula de dimensió “**productes**”. En aquesta taula tenim una clau primària que està composta per dues variables (id_transaction, id_products) que dona la garantia que cada registre de la taula és únic.

Com hem comentat prèviament, com que el camp “**product_ids**” de la taula “**transactions**” pot contenir diversos id’s de producte, el que farem serà una JSON table per poder passar les dades a la taula intermèdia creada.

Així doncs, primer creem la taula intermèdia “**transactions_products**” amb un CREATE TABLE i les condicions explicades.

```
CREATE TABLE IF NOT EXISTS transactions_products (
    id_transaction VARCHAR(255),
    id_products VARCHAR(20),
    PRIMARY KEY (id_transaction, id_products),
    FOREIGN KEY (id_transaction) REFERENCES transactions(id_transaction),
    FOREIGN KEY (id_products) REFERENCES products(product_id)
);
```

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:27:33	CREATE TABLE IF NOT EXISTS transactions_products (id_transaction VARCHAR(255), id_products VARCHAR(20), PRIMARY KEY (id_tr...	0 row(s) affected	0.031 sec

Un cop creada la taula, introuïm les dades en la taula a través d’un INSERT INTO, fent una consulta on generarem la JSON TABLE per a poder anar separant els productes de la variable “product_ids” de la taula “transaccions”.

Primerament, transformarem la cadena de text del camp “product_ids” en una **vector** JSON amb la instrucció **CONCAT('["', REPLACE(t.product_ids, ',', '","'), '"]')** , generant una expressió per comes que es transformada posant-ho entre cometes dobles i ho concatena tot. Això ja seria un tipus de vector acceptat per JSON‘["“1","2","3"]'. Després, amb la instrucció, **$[*]' COLUMNS(product_ids VARCHAR(20) PATH '$')** ho portem tot a una taula temporal que anomenem **jst** i ho converteix en una taula que descomposa una llista de “**product_id**” amb un “**product_id**” per fila.

Després **es genera una fila per cada producte de cada transacció**, a través de un JOIN (implícit) entre **transactions** t i la taula JASON **jst**.

```
INSERT INTO transactions_products (id_transaction, id_products) -- introduirem les dades del SELECT
SELECT  t.id_transaction, TRIM(jst.product_ids) AS id_product
FROM transactions t,
   JSON_TABLE(
        CONCAT('["', REPLACE(t.product_ids, ',', '","'), '"]'),
        '$[*]' COLUMNS(product_ids VARCHAR(20) PATH '$') -- transforma l’array JSON en files d’una taula temporal.
    ) AS jst -- genera un Join implícit entre FT i JST, generant una fila per cada producte de cada transacció.
WHERE t.product_ids IS NOT NULL AND t.product_ids <> '';

##Visualitzem la taula transaction_products

SELECT *
FROM transactions_products;
```

Result Grid

id_transaction	id_products
001A60EA-DC9C-4E5A-9460-6628B100E7E1	1
0032F0BB-B8E6-4AA5-B5EE-EEAD533C0C48	1
0034238 1-503D-422D-B5A8-F2D4FFAAD4C7	1
004C0A80-E537-46D8-BE44-343D2176DF15	1
004D1DB5-B2CB-4460-98B6-31C42CA96E5F	1
0062599C-0A1F-4405-AD55-45E20043B551	1
007B1297-C3ED-4966-802C-68FE106AD25C	1
007EA15D-AA1F-4FB3-9483-7D05BE99CEC0	1
00967BD1-1849-47C1-98E8-8F6C0EA5DF32	1
0096A25B-8DAD-47DE-85AD-BA374BE7736	1
00C61197-05FF-4ECD-A523-E0D6E2E30DDD	1
00E800C8-F09E-4662-88A4-6D16DF32F0E5	1
00E9A7E4-9FA1-42EC-85BC-ECBE44AAE558	1

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	10:28:32	INSERT INTO transactions_products (id_transaction, id_products) -- introduirem les dades del SELECT SELECT t.id_transaction, TRIM(jst.product_i...	253391 row(s) affected Records: 253391 Duplicates: 0 Warnings: 0	14.484 sec
2	10:28:52	SELECT * FROM transactions_products LIMIT 0, 50000	50000 row(s) returned	0.000 sec / 0.000 sec

Un cop feta la taula intermèdia, podem fer la consulta per saber quantes vegades s'ha venut cada producte. Fem servir JOIN per unir productes amb la taula intermèdia "**transactions_products**", fent COUNT en el SELECT per comptar en quantes transaccions apareix cada producte. Agrupem per producte amb GRUOP BY i ordenem amb ORDER BY de més a menys venut.

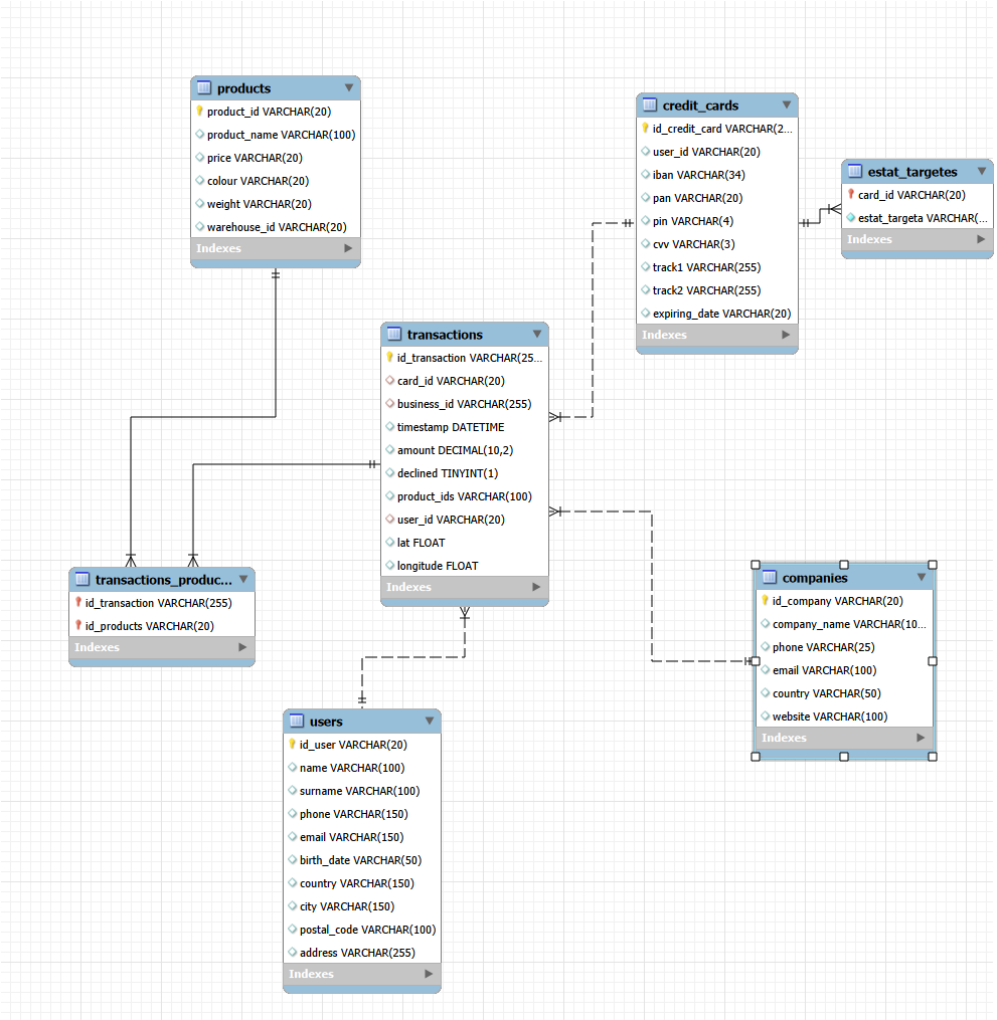
```
-- Calculem el nombre de vegades que s'ha venut cada producte.
SELECT p.product_id,p.product_name, COUNT(tp.id_transaction) AS vegades_venut
FROM products p
JOIN transactions_products tp ON p.product_id = tp.id_products
GROUP BY p.product_id
ORDER BY vegades_venut DESC;
```

product_id	product_name	vegades_venut
52	riverlands the duel	2654
29	Tully maester Tarly	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2601
87	sith Jade	2598
48	rock Renly in	2597
33	duel warden	2597
23	riverlands north	2593
68	Stark Karstark	2589
88	Stannis warden so...	2587
4	warden south duel	2584
73	Dorne bastard	2584
28	chewbacca mustafar	2584
31	Lannister	2580
34	of north	2580
38	ewok sith	2578
54	dooku solo	2575
61	Winterfell Lannister	2575
36	Lannister	2575
24	south duel tourney	2574
11	Karstark Dorne	2573
72	north	2573
2	Tarly Stark	2573
10	Karstark Dorne	2571
39	warden	2571
47	Tully	2571
74	duel warden	2569
64	Direwolf	2568

Result 25

Output

Action Output

#	Time	Action	Message	Duration / Fetch
1	09:25:15	SELECT p.product_id,p.product_name, COUNT(tp.id_transaction) AS vegades_venut FROM products p JOIN transactions_products tp ON p.produ...	100 row(s) returned	0.140 sec / 0.000 sec

El model que ha quedat un cop realitzat els exercicis, és el següent: